

OPERATIONS REASERCH Project

Algorithmes de flots dans les graphes

Amir Benyahia

Master Informatique 1^{re} année

Mai 2026

Table des matières

1	Introduction	1
2	Structure de données : le graphe résiduel	1
2.1	Définition	1
2.2	Représentations possibles	1
2.3	Couplage forward/backward	2
2.4	Exemple d'évolution	3
2.5	Pourquoi cette structure marche bien	3
3	Ford-Fulkerson (Edmonds-Karp)	3
3.1	Principe	3
3.2	Pourquoi BFS et pas DFS	3
3.3	Min cut	4
4	Min Cost Flow	5
4.1	Successive Shortest Augmenting Paths	5
4.2	Variante Bellman-Ford	5
4.3	Variante Dijkstra avec renormalisation	6
4.4	Vérification croisée	7
5	Détection de cycles négatifs	7
5.1	Pourquoi	7
5.2	Algorithme	7
5.3	Intégration dans le min cost flow	8
6	Visualisation et CLI	8
6.1	Format du fichier d'entrée	8
6.2	Visualiseur Graphviz	9
6.3	CLI	9
7	Tests	10
8	Conclusion	10

1 Introduction

Dans ce projet j'ai codé en Python, sans librairie de graphes, quatre algorithmes de flots :

- Ford-Fulkerson (variante Edmonds-Karp) avec extraction de la coupe minimale
- Min cost flow avec Bellman-Ford
- Min cost flow avec Dijkstra et renormalisation des coûts
- Détection de cycles négatifs dans le graphe résiduel

J'utilise uniquement les structures de la lib standard (`dict`, `list`, `deque`, `heapq`). Le projet fait à peu près 600 lignes de Python, avec une CLI, un visualiseur Graphviz et trois fichiers d'exemple.

J'ai commencé par Ford-Fulkerson qui était le plus simple, ensuite le min cost flow avec Bellman-Ford, et enfin la version Dijkstra. C'est la dernière qui m'a pris le plus de temps. Le concept de renormalisation est passé une fois, mais j'ai eu plusieurs bugs à cause des potentiels que je mettais mal à jour entre les itérations.

2 Structure de données : le graphe résiduel

Tous mes algorithmes utilisent la même structure de graphe résiduel.

2.1 Définition

Soit $G = (V, A)$ un graphe orienté avec une capacité $u(i, j) \geq 0$, un coût $c(i, j)$ et une borne inférieure $l(i, j) \geq 0$ sur chaque arc. Soit $f : A \rightarrow \mathbb{N}$ un flot vérifiant $l(i, j) \leq f(i, j) \leq u(i, j)$.

Le graphe résiduel G_f contient pour chaque arc original (i, j) :

- un arc forward (i, j) de capacité résiduelle $r_f(i, j) = u(i, j) - f(i, j)$ et de coût $c(i, j)$
- un arc backward (j, i) de capacité résiduelle $r_b(j, i) = f(i, j) - l(i, j)$ et de coût $-c(i, j)$

Le coût négatif sur l'arc backward sert à « rembourser » le coût quand on annule du flot. Si on a payé c pour envoyer une unité, annuler cette unité doit nous rendre c , sinon les calculs sont faux.

À tout moment :

$$r_f(i, j) + r_b(j, i) = u(i, j) - l(i, j)$$

2.2 Représentations possibles

J'ai regardé trois options avant de choisir.

Matrice d'adjacence ($V \times V$). Simple à indexer mais $O(V^2)$ en mémoire même quand le graphe est creux. Et il faut stocker capacité, flot, coût et borne inférieure pour chaque case, ça devient lourd. Pas retenu.

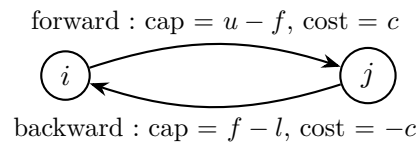
Dict de dicts (`{u: {v: cap}}`). Compact pour les graphes creux mais l'arc inverse (v, u) doit être stocké à part, et il faut maintenir la cohérence entre les deux à la main. Source de bugs.

Liste d'adjacence avec arcs couplés. Pour chaque arc original on crée deux objets `Arc` (forward et backward) reliés par un champ `reverse`. C'est ce que j'ai gardé.

La structure finale est `dict[node → list[Arc]]`. J'utilise un `dict` alors qu'une liste indexée aurait suffi (les nœuds sont numérotés de 0 à $n - 1$), mais ça ne change rien en pratique.

2.3 Couplage forward/backward

Pour chaque arc original (i, j) avec capacité u , coût c et borne inférieure l :



Les deux arcs se pointent mutuellement :

`forward.reverse = backward,      backward.reverse = forward`

```

1  class Arc:
2      def __init__(self, src, dst, capacity, cost=0, lower=0):
3          self.src = src
4          self.dst = dst
5          self.capacity = capacity
6          self.cost = cost
7          self.lower = lower
8          self.flow = 0
9          self.reverse = None
10
11
12  class ResidualGraph:
13      def __init__(self, num_nodes):
14          self.num_nodes = num_nodes
15          self.adj = {i: [] for i in range(num_nodes)}
16
17      def add_arc(self, src, dst, capacity, cost=0, lower=0):
18          forward = Arc(src, dst, capacity - lower, cost, lower)
19          backward = Arc(dst, src, 0, -cost, 0)
20          forward.reverse = backward
21          backward.reverse = forward
22          self.adj[src].append(forward)
23          self.adj[dst].append(backward)
24          return forward
25
26      def augment(self, path_arcs, flow_amount):
27          for arc in path_arcs:
28              arc.capacity -= flow_amount
29              arc.reverse.capacity += flow_amount
30              arc.flow += flow_amount
31              arc.reverse.flow -= flow_amount

```

Listing 1 – Classes `Arc` et `ResidualGraph`

Le champ `capacity` c'est la capacité résiduelle qui change à chaque augmentation. Le champ `flow` stocke le flot en cours sur l'arc. C'est redondant avec `capacity` mais ça aide pour l'affichage et la visualisation. La capacité originale se retrouve par `arc.capacity + arc.flow`.

Au début j'avais oublié de mettre à jour `flow` sur l'arc backward, et la sortie du visualiseur montrait des arcs en double. C'est comme ça que je m'en suis aperçu.

2.4 Exemple d'évolution

Soit un arc (i, j) avec $u = 10$, $c = 4$, $l = 0$.

État initial :

- forward : `capacity = 10`, `flow = 0`
- backward : `capacity = 0`, `flow = 0`

Après augmentation de 3 unités sur le forward :

- forward : `capacity = 7`, `flow = 3`
- backward : `capacity = 3`, `flow = -3`

L'invariant tient bien : $7 + 3 = 10$. On peut maintenant envoyer encore 7 unités sur le forward, ou en annuler 3 via le backward.

2.5 Pourquoi cette structure marche bien

Trois raisons.

Mise à jour en $O(1)$ par arc. `augment` fait quatre affectations, sans recherche ni table auxiliaire.

Itération naturelle des voisins. Avec `for arc in graph.adj[node]` on parcourt les sortants. BFS, Dijkstra et Bellman-Ford utilisent tous la même API.

Pas besoin de reconstruire le résiduel. La capacité résiduelle est dans `arc.capacity` et évolue au fil des augmentations. La structure reste la même tout du long.

Du coup, chaque algorithme se résume à : trouver un chemin, calculer le bottleneck, appeler `augment`.

3 Ford-Fulkerson (Edmonds-Karp)

3.1 Principe

Tant qu'il y a un chemin de la source au puits dans le graphe résiduel, on l'augmente. Quand il n'y en a plus, le flot est maximum.

3.2 Pourquoi BFS et pas DFS

L'algo original ne dit pas comment trouver le chemin. Avec DFS, sur le contre-exemple classique (capacités énormes et un arc de capacité 1 au milieu), on peut faire un nombre exponentiel d'itérations. Avec BFS on prend le chemin le plus court en nombre d'arcs, c'est la variante Edmonds-Karp qui tourne en $O(VE^2)$ peu importe les capacités.

```

1 def bfs_find_path(graph, source, sink):
2     visited = {source: None}
3     queue = deque([source])
4
5     while queue:
6         node = queue.popleft()
7         if node == sink:
8             path_arcs = []
9             current = sink
10            while visited[current] is not None:
11                arc = visited[current]
12                path_arcs.append(arc)
13                current = arc.src
14            path_arcs.reverse()
15            bottleneck = min(arc.capacity for arc in path_arcs)
16            return path_arcs, bottleneck
17
18        for arc in graph.adj[node]:
19            if arc.dst not in visited and arc.capacity > 0:
20                visited[arc.dst] = arc
21                queue.append(arc.dst)
22
23    return None, 0

```

Listing 2 – BFS dans le graphe résiduel

```

1 def ford_fulkerson(graph, source, sink):
2     max_flow_value = 0
3     while True:
4         path_arcs, bottleneck = bfs_find_path(graph, source, sink)
5         if path_arcs is None or bottleneck == 0:
6             break
7         graph.augment(path_arcs, bottleneck)
8         max_flow_value += bottleneck
9     return max_flow_value

```

Listing 3 – Boucle principale

3.3 Min cut

Le théorème max-flow / min-cut dit que la valeur du flot max égale la capacité min d'une coupe entre source et puits. Une fois Ford-Fulkerson terminé :

1. BFS dans le résiduel à partir de la source
2. S = nœuds atteints, $T = V \setminus S$
3. la coupe = arcs saturés qui vont de S vers T

Sortie sur `examples/simple.txt` (5 nœuds, source 0, puits 4) :

=== FORD-FULKERSON ===

Flot max : 15

Min Cut :

S = {0, 1, 2, 3}

Arcs coupés : (3→4, cap=15)

Capacité totale : 15 égale au flot max

4 Min Cost Flow

Maintenant chaque arc a un coût $c(i, j)$ par unité de flot. On cherche le flot qui minimise le coût total.

4.1 Successive Shortest Augmenting Paths

À chaque itération, au lieu de prendre n'importe quel chemin augmentant comme dans Ford-Fulkerson, on prend le plus court chemin en termes de coût dans le résiduel et on l'augmente. Tant qu'on prend les chemins les plus courts, le résiduel ne contient pas de cycle négatif, ce qui correspond à la condition d'optimalité du flot :

$$\forall (u, v) \in A_f, \quad d(v) \leq d(u) + c(u, v)$$

Reste à choisir comment trouver le plus court chemin.

4.2 Variante Bellman-Ford

Bellman-Ford marche avec n'importe quels coûts. Il fait $n - 1$ passes de relaxation et donne la distance correcte tant qu'il n'y a pas de cycle négatif. Comme le résiduel a des arcs backward de coût $-c$, on a forcément des coûts négatifs. Bellman-Ford les gère, donc pas besoin de transformer le graphe.

```

1 def bellman_ford_shortest_path(graph, source, sink):
2     INF = float('inf')
3     dist = {node: INF for node in graph.adj}
4     pred = {node: None for node in graph.adj}
5     dist[source] = 0
6
7     n = graph.num_nodes
8     for _ in range(n - 1):
9         updated = False
10        for node in graph.adj:
11            if dist[node] == INF:
12                continue
13            for arc in graph.adj[node]:
14                if arc.capacity > 0:
15                    new_dist = dist[node] + arc.cost
16                    if new_dist < dist[arc.dst]:
17                        dist[arc.dst] = new_dist
18                        pred[arc.dst] = arc
19                updated = True
20        if not updated:

```

```

21         break
22     return dist, pred

```

Listing 4 – Bellman-Ford

Complexité : $O(VE)$ par itération, et au plus F augmentations où F est la valeur du flot final, donc $O(F \cdot VE)$ en tout.

4.3 Variante Dijkstra avec renormalisation

Dijkstra est plus rapide que Bellman-Ford : $O((V+E) \log V)$ avec un tas binaire. Mais il ne marche pas avec des coûts négatifs, et le résiduel en a forcément. La solution c'est la renormalisation à la Johnson : on introduit un potentiel $h[v]$ par nœud et un coût réduit

$$c_R(u, v) = h[u] + c(u, v) - h[v]$$

Si on choisit bien h , on a $c_R(u, v) \geq 0$ pour tous les arcs résiduels et Dijkstra peut marcher.

4.3.1 Initialisation

Au départ on prend $h[v]$ = distance Bellman-Ford de la source à v . Cette distance vérifie l'inégalité triangulaire dans le résiduel :

$$h[v] \leq h[u] + c(u, v) \quad \Leftrightarrow \quad h[u] + c(u, v) - h[v] \geq 0$$

4.3.2 Mise à jour des potentiels

À chaque itération, après avoir calculé les distances $d(v)$ par Dijkstra (en coûts réduits), on fait

$$h[v] \leftarrow h[v] + d(v)$$

Avec ça, les nouveaux potentiels gardent $c_R \geq 0$ pour tous les arcs résiduels du tour suivant, y compris les arcs qui apparaissent à cause de l'augmentation.

C'est exactement cette mise à jour qui m'a planté plusieurs fois. À la première version j'avais oublié d'ajouter $d(v)$ à $h[v]$ entre les itérations, donc à la deuxième augmentation Dijkstra tombait sur des coûts réduits négatifs et le `RuntimeError` se déclenchait. Une fois que j'ai corrigé, les résultats matchaient ceux de Bellman-Ford.

4.3.3 Code

L'extrait important de Dijkstra avec coûts réduits :

```

1  for arc in graph.adj[node]:
2      if arc.capacity <= 0:
3          continue
4      reduced_cost = potentials[arc.src] + arc.cost - potentials[arc.dst]
5      if reduced_cost < -1e-9:
6          raise RuntimeError(...)

```



```

7   new_dist = dist[node] + max(0.0, reduced_cost)
8   if new_dist < dist[arc.dst]:
9       dist[arc.dst] = new_dist
10      pred[arc.dst] = arc
11      heapq.heappush(heap, (new_dist, arc.dst))

```

Listing 5 – Cœur de Dijkstra avec coûts réduits

Le `raise RuntimeError` sur les coûts réduits négatifs c'est mon garde-fou : si ça se déclenche, c'est que la mise à jour des potentiels est cassée. C'est ce qui m'a permis de déboguer.

4.3.4 Complexités

	Par itération	Total
Bellman-Ford	$O(V \cdot E)$	$O(F \cdot V \cdot E)$
Dijkstra (potentiels)	$O((V + E) \log V)$	$O(F \cdot (V + E) \log V)$

F est borné par la valeur du flot maximum.

4.4 Vérification croisée

Sur un même graphe, les deux variantes doivent donner le même résultat. Sur `examples/simple.txt` :

```

=== MIN COST FLOW (Bellman-Ford) ===
Flot total : 15
Coût total : 60

=== MIN COST FLOW (Dijkstra) ===
Flot total : 15
Coût total : 60

```

C'est ma meilleure preuve pratique que la renormalisation est correcte.

5 Détection de cycles négatifs

5.1 Pourquoi

Un flot f est optimal si et seulement si le résiduel G_f n'a aucun cycle négatif. Tant qu'on peut trouver un cycle de coût < 0 , on peut renvoyer du flot autour et baisser le coût total.

Ça sert aussi pour les entrées pathologiques : si le graphe initial a déjà un cycle négatif, on lève une erreur claire au lieu de renvoyer un résultat faux.

5.2 Algorithme

L'idée vient encore de Bellman-Ford. Après $n - 1$ itérations les distances sont stables s'il n'y a pas de cycle négatif. Si à la n -ième itération on peut encore relâcher un arc, c'est qu'il y a un cycle négatif accessible.

```

1  for i in range(n):
2      last_relaxed = None
3      for node in graph.adj:
4          for arc in graph.adj[node]:
5              if arc.capacity <= 0:
6                  continue
7              if dist[node] + arc.cost < dist[arc.dst]:
8                  dist[arc.dst] = dist[node] + arc.cost
9                  pred[arc.dst] = arc
10                 last_relaxed = arc.dst
11
12  if last_relaxed is None:
13      return False, None
14  # sinon on remonte les predecesseurs pour reconstruire le cycle

```

Listing 6 – Boucle principale de détection

J’initialise $\text{dist}[v] = 0$ pour tous les nœuds (et pas $+\infty$ sauf pour la source). Ça permet de détecter les cycles négatifs même quand ils ne sont pas atteignables depuis un point fixe. Pour reconstruire le cycle, je pars du dernier nœud relaxé, je remonte n fois les prédécesseurs pour être sûr d’être dans le cycle, puis je suis les arcs jusqu’au point de départ.

5.3 Intégration dans le min cost flow

Dans la version Bellman-Ford du min cost flow, j’appelle `detect_negative_cycle` à chaque itération avant de chercher un nouveau chemin. Si un cycle négatif apparaît, l’algo lève une exception au lieu de boucler.

Dans la version Dijkstra le test n’est fait qu’une fois au début. La mise à jour des potentiels garantit qu’aucun cycle négatif ne peut apparaître pendant les itérations suivantes.

6 Visualisation et CLI

6.1 Format du fichier d’entrée

```

# Commentaire
nodes 5
arc 0 1 10 2      # src dst capacite cout
arc 0 2 8 4
arc 1 3 10 1
arc 2 3 6 3
arc 3 4 15 1
source 0
sink 4

```

Quatre mots-clés : `nodes`, `arc`, `source`, `sink`. Les commentaires commencent par `#`. Le coût est optionnel (par défaut 0). Du coup je peux décrire un graphe à la main en quelques lignes, c’est largement plus simple que JSON ou autre.

6.2 Visualiseur Graphviz

Au début je voulais juste afficher les résultats en console, mais sur les graphes un peu plus gros (genre l'affectation Peter/Paul/Mary) c'était illisible. J'ai donc ajouté un visualiseur qui génère une image PNG. Le cours en parle d'ailleurs slide 1.12 avec le Dot Language, donc autant l'utiliser.

Le visualiseur fait deux choses :

1. génère un fichier `.dot` avec la description du graphe (nœuds, arcs, labels)
2. appelle `dot -Tpng` en sous-processus pour produire le PNG

Le code couleur sur les arcs :

- **rouge** : arc saturé (`flot == capacité`) — c'est un goulot d'étranglement, en général il fait partie de la min cut
- **vert** : arc utilisé (`flot > 0` mais pas saturé)
- **noir** : arc inutilisé (`flot == 0`)

Chaque arc affiche `flot/capacité`, `coût`. C'est le format que j'ai vu dans le cours et c'est pratique parce qu'on voit d'un coup d'œil l'état complet du graphe après l'algorithme.

Sur `simple.txt` après Ford-Fulkerson, on voit que les arcs sortants de la source sont en vert (en cours d'utilisation) et que l'arc final ($3 \rightarrow 4$) ressort en rouge. Du coup on sait sans réfléchir que la min cut passe par lui — et c'est confirmé par la sortie console.

C'est aussi grâce au visualiseur que j'ai trouvé un bug au début. Sur un de mes premiers tests, j'avais des arcs qui apparaissaient en double sur l'image, avec des valeurs bizarres. C'est ce qui m'a fait remonter au fait que je ne mettais pas à jour `flow` sur l'arc backward dans `augment`. Sans le PNG j'aurais sans doute mis plus de temps à tomber dessus.

6.3 CLI

```
python main.py examples/simple.txt --algo ford_fulkerson
python main.py examples/simple.txt --algo min_cost_bf --visualize
python main.py --demo
```

J'ai utilisé `argparse` pour gérer la ligne de commande. Il y a trois flags :

- un fichier `.txt` en entrée et `--algo` pour choisir l'algo (`ford_fulkerson`, `min_cost_bf` ou `min_cost_dijkstra`)
- `--visualize` pour générer l'image PNG dans `outputs/`
- `--demo` qui lance les démos hardcodées (Ford-Fulkerson, graphe d'affectation Peter/Paul/Mary, comparaison BF vs Dijkstra, détection de cycles)

Les démos servaient surtout pendant le développement pour tester rapidement après une modif. Je les ai laissées parce qu'elles permettent de vérifier en une commande que tout fonctionne sans

avoir besoin de fichier d'entrée.

7 Tests

J'ai écrit 9 tests pytest répartis en trois fichiers (un par algorithme). C'est pas demandé dans l'énoncé mais comme je modifiais souvent le code, j'avais besoin d'un filet de sécurité pour pas casser les algos déjà écrits en travaillant sur les autres.

Ce que les tests vérifient :

Pour Ford-Fulkerson (3 tests) : que le max flow renvoyé est correct sur des graphes dont je connais la réponse, et surtout que la capacité de la min cut est bien égale au max flow. C'est le théorème max-flow / min-cut, donc si l'égalité ne tient pas, c'est qu'il y a un bug soit dans `ford_fulkerson` soit dans `find_min_cut`.

Pour le min cost flow (3 tests) : que Bellman-Ford et Dijkstra donnent le même résultat sur le même graphe. C'est ma vérification croisée la plus utile : si les deux algos divergent, c'est forcément un bug dans l'un des deux. Quand j'ai eu des problèmes avec la mise à jour des potentiels, c'est ce test qui a planté en premier.

Pour la détection de cycles négatifs (3 tests) : qu'un cycle de coût -3 est bien détecté, qu'un cycle de coût $+1$ ne l'est pas (pas de faux positif), et que la fonction `assert_no_negative_cycle` lève bien une exception quand il faut.

Les tests tournent en environ 0.05 seconde. C'est négligeable, donc je les lance avant chaque commit avec `python -m pytest tests/ -v`.

8 Conclusion

Le projet repose principalement sur la structure du graphe résiduel avec arcs couplés. Une fois qu'elle est en place, les algorithmes sont assez directs à coder. La partie qui m'a donné le plus de mal c'est Dijkstra avec les potentiels : si on rate la mise à jour de h entre les itérations, l'algo renvoie des distances fausses sans rien dire. C'est pour ça que j'ai mis le `raise RuntimeError` sur les coûts réduits négatifs, pour qu'au moins ça plante au lieu de me sortir un résultat incorrect.

Si j'avais plus de temps, j'aimerais bien essayer le min cost flow par cancellation de cycles négatifs (l'autre méthode du cours) pour comparer avec celle des chemins augmentants.